

Progetto di Laboratorio

AREA HADAMARD GATE

Implementazione VHDL di un gate di Hadamard 2x2
su FPGA Boolean Board (Spartan-7)

Corso: **Introduzione al Trattamento Quantistico delle Informazioni**
Sezione 1 del Report: **Listato di codice e spiegazione dettagliata**

Documento generato automaticamente con ReportLab

1. Sezione 1 — Listato di codice e spiegazione dettagliata

1.1 Inquadramento teorico

Il gate di Hadamard è la porta quantistica a 1-qubit più importante nei circuiti di Deutsch–Jozsa, Grover e Shor: trasforma uno stato di base in una sovrapposizione bilanciata. La sua matrice è:

$$H = (1/\sqrt{2}) * \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

Applicata al vettore di stato $|\psi\rangle = [a; b]^T$ produce:

$$H * |\psi\rangle = (1/\sqrt{2}) * \begin{bmatrix} a+b \\ a-b \end{bmatrix}$$

L'obiettivo del progetto è realizzare questa trasformazione in hardware su FPGA, usando aritmetica a virgola fissa a **12 bit con segno** e **8 bit di parte frazionaria** (formato **Q3.8**: 1 bit di segno, 3 bit interi, 8 bit frazionari). Il range rappresentabile è $[-8.000, +7.996]$, con risoluzione $1/256 \approx 3.9 \cdot 10^{-3}$.

La costante $1/\sqrt{2} \approx 0.70710678$ in Q3.8 vale:

$$\text{round}(0.70710678 * 2^8) = 181 = 0000_1011_0101 \text{ (base 2)}$$

1.2 Architettura del progetto

La specifica richiede che **tutti gli ingressi e le uscite della top-level entity siano collegati a registri**. Sono quindi presenti due banchi di registri: uno in ingresso (a_reg, b_reg) e uno in uscita (y0_reg, y1_reg). Il calcolo combinatorio è incapsulato nel modulo **hadamard_core**, riusabile e parametrico. I valori della matrice (1, 1, 1, -1, costante $1/\sqrt{2}$) non sono input ma **costanti**, come permesso dal SUGGERIMENTO della traccia.

```
A_CONST --> [ REG a_reg (12b) ] --\
                                     +--> [ hadamard_core ] --> [ REG y0_reg ] --> y0_obs[5:0]
B_CONST --> [ REG b_reg (12b) ] --/                                     |
                                                                                   \n

hadamard_core:
  sum  = a + b
  diff = a - b
  prod = (...) * 181
  y    = prod >> 8
```

Schema a blocchi della top-level entity con registri I/O.

1.3 Struttura dei file sorgente

File	Ruolo
src/hadamard_core.vhd	Modulo combinatorio: implementa $H \cdot v$
src/hadamard_top.vhd	Top-level: registri I/O, costanti A,B, istanza core
sim/tb_hadamard.vhd	Testbench behavioral con 4 stimoli
constraints/hadamard.xdc	Pin assignment per la Boolean Board

1.4 Listato — hadamard_core.vhd

```
-----
-- File      : hadamard_core.vhd
-- Descrizione : Logica combinatoria del gate di Hadamard 2x2.
--            Calcola  $H * [a;b] = (1/\sqrt{2}) * [a+b ; a-b]$ .
--            Aritmetica a virgola fissa Q3.8 (12 bit signed, 8 frazionari).
-- Target    : Boolean Board (Xilinx Spartan-7 XC7S50)
-----

library ieee;
  use ieee.std_logic_1164.all;
  use ieee.numeric_std.all;

entity hadamard_core is
  generic (
    data_width : integer := 12; -- larghezza totale del dato
    frac_bits  : integer := 8  -- bit di parte frazionaria
  );
  port (
    a_in  : in    signed(data_width - 1 downto 0); -- ampiezza qubit |0>
    b_in  : in    signed(data_width - 1 downto 0); -- ampiezza qubit |1>
    y0_out : out  signed(data_width - 1 downto 0); -- (a+b)/sqrt(2)
    y1_out : out  signed(data_width - 1 downto 0); -- (a-b)/sqrt(2)
  );
end entity hadamard_core;

architecture rtl of hadamard_core is

  -- 1/sqrt(2) in Q3.8: round(0.70710678 * 256) = 181 = "00 0010110101"
  -- Lo rappresentiamo su 12 bit signed (valore positivo).
  constant inv_sqrt2 : signed(data_width - 1 downto 0) :=
    to_signed(181, data_width);

  -- Somma e differenza estese di 1 bit per evitare overflow.
  signal sum_ab  : signed(data_width downto 0);
  signal diff_ab : signed(data_width downto 0);

  -- Prodotti completi: (DATA_WIDTH+1) * DATA_WIDTH = 2*DATA_WIDTH+1 bit
  signal prod_sum  : signed(2 * data_width downto 0);
  signal prod_diff : signed(2 * data_width downto 0);

begin

  -- Somma e differenza con estensione di segno.
  sum_ab <= resize(a_in, data_width + 1) + resize(b_in, data_width + 1);
  diff_ab <= resize(a_in, data_width + 1) - resize(b_in, data_width + 1);

  -- Moltiplicazione per 1/sqrt(2) (Q3.8 * Q3.8 -> Q6.16 esteso).
  prod_sum <= sum_ab * inv_sqrt2;
  prod_diff <= diff_ab * inv_sqrt2;

  -- Riallineamento Q -> Q3.8: scarto FRAC_BITS bit bassi e prendo
  -- DATA_WIDTH bit (con saturazione implicita affidata al range dei segnali).
  -- prod ha 2*DATA_WIDTH+1 bit con 2*FRAC_BITS bit frazionari;
  -- per tornare a FRAC_BITS frazionari devo shiftare a destra di FRAC_BITS.
  y0_out <= resize(shift_right(prod_sum,  frac_bits), data_width);
  y1_out <= resize(shift_right(prod_diff, frac_bits), data_width);

end architecture rtl;
```

Spiegazione dettagliata

Elemento	Cosa fa	Perché è così
library IEEE; use NUMERIC_STD;	Importa i tipi signed/unsigned e operatori aritmetici IEEE.	Standard IEEE consigliato; evita ambiguità di std_logic_arith.
generic DATA_WIDTH, FRAC_BITS	Parametrizza la larghezza del dato e i bit frazionari.	Permette il riuso del modulo per altri formati Q senza riscrivere.

a_in, b_in : signed	Ingressi a 12 bit con segno (Q3.8).	I dati sono in [-8, +8) con risoluzione 1/256.
INV_SQRT2 := to_signed(181, 12)	Costante $1/\sqrt{2}$ in Q3.8.	$\text{round}(0.70710678 \cdot 256) = 181$; errore di quantizzazione $\approx 1.3 \cdot 10^{-4}$.
sum_ab, diff_ab : signed(12)	Operazioni a + b e a - b su 13 bit.	Estensione di 1 bit per evitare overflow quando $ a + b \geq 8$.
prod_sum, prod_diff : signed(24)	Prodotti completi a 25 bit.	$\text{signed}(13) \cdot \text{signed}(12) = \text{signed}(25)$; nessuna perdita di precisione.
shift_right(prod, FRAC_BITS)	Divide per 2^8 , ovvero riallinea il risultato a Q3.8.	signed, shift_right è aritmetico → preserva il segno.
resize(..., DATA_WIDTH)	Ritorna a 12 bit.	I bit alti sono replica del segno per gli ingressi normalizzati.

Aspetto numerico chiave. Il prodotto $\text{signed}(13b) \times \text{signed}(12b)$ produce 25 bit con 16 bit frazionari. Dopo `shift_right(...,8)` restano 17 bit con 8 frazionari; `resize` ne tiene 12. Vivado, per questa moltiplicazione, inferisce un blocco **DSP48E1** per ciascun prodotto (2 totali).

1.5 Listato — hadamard_top.vhd

```
-----
-- File : hadamard_top.vhd
-----

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity hadamard_top is
generic(
    data_width : integer := 12;
    frac_bits   : integer := 8
);
port(
    clk : in std_logic;
    rst : in std_logic;

    -- solo pin realmente disponibili
    y0_obs : out std_logic_vector(5 downto 0);
    y1_obs : out std_logic_vector(5 downto 0)
);
end entity;

architecture rtl of hadamard_top is

    constant a_const : signed(data_width-1 downto 0)
        := to_signed(256,data_width);

    constant b_const : signed(data_width-1 downto 0)
        := to_signed(0,data_width);

    signal a_reg,b_reg : signed(data_width-1 downto 0);

    signal y0_c,y1_c : signed(data_width-1 downto 0);

    signal y0_reg,y1_reg : signed(data_width-1 downto 0);

    component hadamard_core
    generic(
        data_width : integer := 12;
        frac_bits   : integer := 8
    );
    port(
        a_in      : in signed(data_width-1 downto 0);
        b_in      : in signed(data_width-1 downto 0);

        y0_out    : out signed(data_width-1 downto 0);
        y1_out    : out signed(data_width-1 downto 0)
    );
    end component;

begin

    -----
    -- input registers
    -----

    p_in_reg : process(clk)
    begin

        if rising_edge(clk) then

            if rst='1' then
                a_reg<=(others=>'0');
                b_reg<=(others=>'0');
```

```

        else
            a_reg<=a_const;
            b_reg<=b_const;
        end if;

    end if;

end process;

-----
-- core
-----

u_core : hadamard_core
generic map(
    data_width => data_width,
    frac_bits  => frac_bits
)
port map(
    a_in  => a_reg,
    b_in  => b_reg,
    y0_out => y0_c,
    y1_out => y1_c
);

-----
-- output registers
-----

p_out_reg : process(clk)
begin

    if rising_edge(clk) then

        if rst='1' then
            y0_reg<=(others=>'0');
            y1_reg<=(others=>'0');
        else
            y0_reg<=y0_c;
            y1_reg<=y1_c;
        end if;

    end if;

end process;

-----
-- esporta solo bit realmente collegati
-----

y0_obs(0) <= y0_reg(0);
y0_obs(1) <= y0_reg(1);
y0_obs(2) <= y0_reg(4);
y0_obs(3) <= y0_reg(6);
y0_obs(4) <= y0_reg(7);
y0_obs(5) <= y0_reg(11);

y1_obs(0) <= y1_reg(1);
y1_obs(1) <= y1_reg(2);
y1_obs(2) <= y1_reg(3);
y1_obs(3) <= y1_reg(4);
y1_obs(4) <= y1_reg(6);
y1_obs(5) <= y1_reg(7);

end architecture;

```

Spiegazione dettagliata

- 1. Porte fisiche** — clk e rst sono gli unici input osservabili sulla board; il vettore di stato è interno (costante), come consentito dalla traccia. Le uscite top-level sono due bus da 6 bit (*y0_obs*, *y1_obs*) che esportano un sottoinsieme di bit dei registri di uscita, coerentemente con i pin realmente disponibili nel file .xdc. Internamente i segnali di dato restano comunque a 12 bit Q3.8.
- 2. Costanti del vettore stato** — $A_CONST = 256$, $B_CONST = 0 \rightarrow$ in Q3.8 valgono +1.0 e 0.0, cioè lo stato |0■. Cambiando solo queste due righe si testano in hardware tutti gli stati (|1■, |+■, |-■, ...).
- 3. Registri di ingresso p_in_reg** — processo sensibile al solo clk: inferisce flip-flop sincroni con reset sincrono attivo alto. Forzando i registri all'inizio del clock domain, blindiamo il timing del core.
- 4. Istanza u_core** — mappatura diretta dei registri sui port del core. I generic vengono propagati, così il top resta scalabile.
- 5. Registri di uscita p_out_reg** — stesso schema: catturano y0_c/y1_c al fronte di salita. Architetturealmente il core resta quindi incapsulato tra uno stadio di registri di ingresso e uno di uscita.
- 6. Driver delle uscite** — il top non esporta l'intero vettore a 12 bit, ma solo alcuni bit significativi dei registri *y0_reg* e *y1_reg* verso i pin realmente cablati sulla board. Questa scelta non altera la larghezza interna dei dati, che rimane 12 bit signed Q3.8 come richiesto.

1.6 Listato — tb_hadamard.vhd (testbench behavioral)

```
-----
-- File      : tb_hadamard.vhd
-- Descrizione : Testbench BEHAVIORAL del gate di Hadamard 2x2.
--            Verifica 4 tipologie di input:
--            1) Stato |0> = [1.0 ; 0.0]      -> attesa [+0.707 ; +0.707]
--            2) Stato |1> = [0.0 ; 1.0]      -> attesa [+0.707 ; -0.707]
--            3) Stato |+> = [0.707 ; 0.707] -> attesa [+1.0   ; 0.0   ]
--            4) Stato |-> = [0.707 ; -0.707] -> attesa [ 0.0   ; +1.0   ]
-----

library ieee;
  use ieee.std_logic_1164.all;
  use ieee.numeric_std.all;

entity tb_hadamard is
end entity tb_hadamard;

architecture sim of tb_hadamard is

  constant data_width : integer := 12;
  constant frac_bits  : integer := 8;
  constant clk_period : time    := 10 ns;  -- 100 MHz

  -- DUT: istanzio direttamente il core per poter cambiare gli ingressi.
  component hadamard_core is
    generic (
      data_width : integer := 12;
      frac_bits  : integer := 8
    );
    port (
      a_in  : in    signed(DATA_WIDTH - 1 downto 0);
      b_in  : in    signed(DATA_WIDTH - 1 downto 0);
      y0_out : out  signed(DATA_WIDTH - 1 downto 0);
      y1_out : out  signed(DATA_WIDTH - 1 downto 0)
    );
  end component hadamard_core;

  signal a_in, b_in : signed(data_width - 1 downto 0) := (others => '0');
  signal y0, y1    : signed(data_width - 1 downto 0);

  -- Funzione di utilita': converte un real (numero in virgola mobile)
  -- nella sua rappresentazione Q3.8 a 12 bit signed.

  function to_q (
    val : real
  ) return signed is
  begin

    return to_signed(integer(val * real(2 ** FRAC_BITS)), DATA_WIDTH);

  end function to_q;

  -- Procedura di check con tolleranza (LSB).

  procedure check (
    constant name : string;
    signal  got    : signed;
    constant exp  : real
  ) is

    variable exp_q : signed(DATA_WIDTH - 1 downto 0);
    variable diff  : integer;

  begin

    exp_q := to_q(exp);
```

```

diff := to_integer(got) - to_integer(exp_q);

if (diff < 0) then
    diff := -diff;
end if;

if (diff <= 2) then
    report name & " OK (got=" & integer'image(to_integer(got)) &
        " exp=" & integer'image(to_integer(exp_q)) & ")"
        severity note;
else
    report name & " FAIL (got=" & integer'image(to_integer(got)) &
        " exp=" & integer'image(to_integer(exp_q)) & ")"
        severity error;
end if;

end procedure check;

begin

-----
-- Istanza DUT
-----
dut : component hadamard_core
generic map (
    data_width => DATA_WIDTH, frac_bits => FRAC_BITS
)
port map (
    a_in  => a_in,
    b_in  => b_in,
    y0_out => y0,
    y1_out => y1
);

-----
-- Stimoli
-----
stim_proc : process is
begin

-----
-- Test 1:  $|0\rangle = [1 \ ; \ 0] \rightarrow [1/\sqrt{2} \ ; \ 1/\sqrt{2}] \sim [+0.707 \ ; \ +0.707]$ 
-----
a_in <= to_q(1.0);
b_in <= to_q(0.0);
wait for clk_period;
check("TEST1  $|0\rangle$  y0", y0, 0.70710678);
check("TEST1  $|0\rangle$  y1", y1, 0.70710678);

-----
-- Test 2:  $|1\rangle = [0 \ ; \ 1] \rightarrow [1/\sqrt{2} \ ; \ -1/\sqrt{2}]$ 
-----
a_in <= to_q(0.0);
b_in <= to_q(1.0);
wait for clk_period;
check("TEST2  $|1\rangle$  y0", y0, 0.70710678);
check("TEST2  $|1\rangle$  y1", y1, -0.70710678);

-----
-- Test 3:  $|+\rangle = [0.707 \ ; \ 0.707] \rightarrow [1.0 \ ; \ 0.0]$ 
-----
a_in <= to_q(0.70710678);
b_in <= to_q(0.70710678);
wait for clk_period;
check("TEST3  $|+\rangle$  y0", y0, 1.0);
check("TEST3  $|+\rangle$  y1", y1, 0.0);

-----

```

```

-- Test 4: |-> = [0.707 ; -0.707] -> [0.0 ; 1.0]
-----
a_in <= to_q(0.70710678);
b_in <= to_q(-0.70710678);
wait for clk_period;
check("TEST4 |-> y0", y0, 0.0);
check("TEST4 |-> y1", y1, 1.0);

report "Simulazione completata."
  severity note;
wait;

end process stim_proc;

end architecture sim;

```

Spiegazione dettagliata

Entità senza port — convenzione standard per un testbench: non si interfaccia col mondo esterno, contiene solo stimoli e DUT.

Funzione to_q(real) — converte un numero in virgola mobile nel suo equivalente Q3.8. È usata solo in simulazione (non sintetizzabile) per rendere leggibili i test, p.es. to_q(0.707) invece di to_signed(181,12).

Procedura check — confronta il valore osservato con quello atteso ammettendo un errore massimo di 2 LSB (≈ 0.0078). La tolleranza serve perché (1) la costante $INV_SQRT2 = 181/256$ è essa stessa quantizzata e (2) il prodotto seguito da shift_right introduce un troncamento.

4 stimoli — sono i 4 stati di base più rappresentativi per validare anche l'**involutività** di H ($H^2 = I$):

#	Ingresso [a;b]	Stato in	Uscita attesa [y0;y1]	Stato out
1	[+1.000; 0.000]	0■	[+0.707; +0.707]	+■
2	[0.000; +1.000]	1■	[+0.707; -0.707]	-■
3	[+0.707; +0.707]	+■	[+1.000; 0.000]	0■
4	[+0.707; -0.707]	-■	[0.000; +1.000]	1■

1.7 Listato — constraints/hadamard.xdc

```
## =====
## hadamard.xdc
## Spartan-7 Boolean Board
## Device: xc7s50csga324-1
## =====

## -----
## CLOCK 100 MHz
## -----
set_property PACKAGE_PIN F14 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]

create_clock -name sys_clk -period 10.000 [get_ports clk]

## -----
## RESET
## -----
set_property PACKAGE_PIN J2 [get_ports rst]
set_property IOSTANDARD LVCMOS33 [get_ports rst]

## -----
## y0_obs[0..5]
## -----
set_property PACKAGE_PIN A8 [get_ports {y0_obs[0]}]
set_property PACKAGE_PIN A9 [get_ports {y0_obs[1]}]
set_property PACKAGE_PIN R7 [get_ports {y0_obs[2]}]
set_property PACKAGE_PIN T5 [get_ports {y0_obs[3]}]
set_property PACKAGE_PIN R6 [get_ports {y0_obs[4]}]
set_property PACKAGE_PIN T11 [get_ports {y0_obs[5]}]

## -----
## y1_obs[0..5]
## -----
set_property PACKAGE_PIN U12 [get_ports {y1_obs[0]}]
set_property PACKAGE_PIN V12 [get_ports {y1_obs[1]}]
set_property PACKAGE_PIN V13 [get_ports {y1_obs[2]}]
set_property PACKAGE_PIN V14 [get_ports {y1_obs[3]}]
set_property PACKAGE_PIN U15 [get_ports {y1_obs[4]}]
set_property PACKAGE_PIN V15 [get_ports {y1_obs[5]}]

## -----
## IOSTANDARD
## -----

set_property IOSTANDARD LVCMOS33 [get_ports y0_obs[*]]
set_property IOSTANDARD LVCMOS33 [get_ports y1_obs[*]]
```

Spiegazione dettagliata

create_clock -period 10.000: dichiara il clock di sistema a 100 MHz (Boolean Board), indispensabile perché il timing engine di Vivado calcoli lo slack e quindi $F_{\max}$.

IOSTANDARD LVCMOS33: standard elettrico di default dei banchi I/O della Spartan-7 sulla Boolean Board (3.3 V).

I pin LED riportano in tempo reale lo stato dei registri di uscita y0_reg/y1_reg; per gli ingressi (costanti) non serve allocare pin.

1.8 Riepilogo della Sezione 1

- Il design rispetta integralmente la specifica: top entity con I/O registrati, segnali di dato a 12 bit signed Q3.8, matrice/stato come costanti della top entity (SUGGERIMENTO), target Boolean Board, sintesi Vivado.

- Il modulo è parametrico (DATA_WIDTH, FRAC_BITS) e quindi facilmente estendibile a precisioni maggiori.
- Il calcolo è numericamente corretto: errore massimo ≤ 2 LSB (≈ 0.008 in valore reale), con inferenza automatica di blocchi DSP48 in fase di sintesi.

Nelle prossime sezioni (2 — Linter, 3 — Implementazione, 4 — Testbench) si riporteranno gli output di Vivado e gli screenshot della simulazione behavioral.